Платформено-независими програмни езици

Лекция 3

Наследяване и полиморфизъм

Наследяване, припокриване, класът Object, абстрактни, final и sealed класове, pattern matching

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.



Съдържание на лекцията

1. Наследяване – идея и синтаксис
2. Конструктори, super и полета
3. Припокриване и полиморфизъм
4. Класът Object
5. Абстрактни и final класове
6. Sealed класове и pattern matching

01

Наследяване – идея и синтаксис

Връзката „е“ (is-a) и йерархии от класове

Какво е наследяване?

- Нов клас се създава на базата на съществуващ
- **Наследникът** (подклас) получава полетата и методите на **родителя** (суперклас)
- Наследникът може да:
  - добавя нови полета и методи
  - **припокрива** (override) наследени методи
- Преизползваме код, без да го копираме
- Моделира връзката „**е**“ (is-a): студентът **е** човек

Синтаксис: extends

- `class Наследник extends Родител`
- В Java има **единично** наследяване на класове
- Наследникът вижда `public` и `protected` членовете
- `private` членовете съществуват, но не са достъпни директно

```
public class Person {
    protected String name;

    public Person(String name) {
        this.name = name;
    }

    public String greet() {
        return "Здравей, аз съм " + name;
    }
}

public class Student extends Person {
    private int fn;
    // ... конструктор и методи
}
```

Използване на наследник

- Обектът на наследника има всички членове на родителя
- Наследникът добавя свои методи
- Родителят **не знае** за наследниците си

```
public class Student extends Person {
    private int fn;

    public Student(String name, int fn) {
        super(name);
        this.fn = fn;
    }

    public int getFn() { return fn; }
}

Student s = new Student("Ана", 12345);
System.out.println(s.greet());  // наследен
System.out.println(s.getFn());  // собствен
```

Йерархия на класовете в Java

- Всеки клас има точно един пряк родител
- Ако не е указан `extends` – родител е `java.lang.Object`
- `Object` е **коренът** на йерархията
- Пример от JDK: `Object` → `Number` → `Integer`, `Double`, `BigDecimal`
- Пример: `Object` → `Throwable` → `Exception` → `IOException`
- Колкото по-надолу, толкова по-**специализиран** е класът

Наследяване в Python и в Java

Python

- `class Student(Person):`
- **множествено** наследяване на класове
- `super().__init__(name)`
- всички методи могат да се припокрият
- няма `abstract` в езика – модул `abc`

Java

- `class Student extends Person`
- **единично** наследяване на класове
- `super(name)` – първи в конструктора
- `final` забранява припокриване
- `abstract` е ключова дума

Пример: базовият клас Employee

- Общото за всички служители
- `protected` поле – достъпно за наследниците
- Метод `salary()`, който наследниците ще припокрият

```
public class Employee {
    private final String name;
    protected double baseSalary;

    public Employee(String name,
                    double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    public String getName() {
        return name;
    }

    public double salary() {
        return baseSalary;
    }
}
```

Пример: наследникът Manager

- Добавя ново поле `bonus`
- Извиква конструктора на родителя
- Ще припокрие `salary()` – по-нататък в лекцията

```
public class Manager extends Employee {
    private double bonus;

    public Manager(String name, double base,
                   double bonus) {
        super(name, base);
        this.bonus = bonus;
    }

    public double getBonus() {
        return bonus;
    }
}
```

Какво може да се прави в наследника?

Действие	Как
да използва наследените полета и методи	директно, ако не са `private`
да декларира ново поле	както във всеки клас
да скрие поле на родителя	поле със същото име (не се препоръчва!)
да припокрие метод на обекта	метод със същата сигнатура + `@Override`
да скрие статичен метод	статичен метод със същата сигнатура
да добави нови методи	както във всеки клас
да извика конструктор на родителя	`super(...)` като първи оператор

Как да изберем модификатора за достъп?

- `private` – за детайлите на реализацията (по подразбиране за полета)
- `protected` – за членове, които наследниците **трябва** да използват
- `public` – за интерфейса към външния свят
- Без модификатор – за помощни класове вътре в пакета
- Внимание: `protected` полета обвързват наследниците с реализацията на родителя
- По-добре: `private` поле + `protected` метод за достъп

protected достъп – пример

- `protected` поле е достъпно в наследника
- По-добре: `private` поле + `protected` метод
- Така родителят контролира промяната

```
public class Employee {
    private double baseSalary;

    protected void raise(double pct) {
        if (pct > 0 && pct <= 50) {
            baseSalary *= 1 + pct / 100;
        }
    }
}

public class Manager extends Employee {
    public void promote() {
        raise(10);           // разрешено
        // baseSalary = 0;  // грешка: private
    }
}
```

02

Конструктори, super и полета

Създаване на обект на наследник

Конструктори и super(...)

- Конструкторите **не се наследяват**
- Конструкторът на наследника първо извиква конструктор на родителя
- `super(...)` – явно извикване; трябва да е първи
- Ако го няма – компилаторът вмъква `super()`
- Ако родителят няма конструктор без параметри → грешка

```
class A {
    A() { System.out.println("A"); }
}

class B extends A {
    B() { System.out.println("B"); }
}

class C extends B {
    C() {
        super();     // неявно и без това
        System.out.println("C");
    }
}

new C();   // A  B  C
```

Гъвкави тела на конструкторите (Java 25)

- Преди Java 25: нищо преди `super(...)`
- От Java 25 може да **проверим аргументите** преди `super(...)`
- Не може да се използва `this` преди `super(...)`

```
public class Manager extends Employee {
    private final double bonus;

    public Manager(String name, double base,
                   double bonus) {
        if (bonus < 0) {                    // Java 25+
            throw new IllegalArgumentException(
                "Бонусът е отрицателен");
        }
        super(name, base);
        this.bonus = bonus;
    }
}
```

Ключовата дума super

- `super(...)` – конструктор на родителя
- `super.метод()` – версията на метода от родителя
- `super.поле` – поле на родителя, ако е скрито
- Позволява да **разширим**, а не да заменим поведението

```
public class Manager extends Employee {
    private final double bonus;

    public Manager(String n, double b,
                   double bonus) {
        super(n, b);
        this.bonus = bonus;
    }

    @Override
    public double salary() {
        return super.salary() + bonus;
    }
}
```

Скриване на полета

- Поле в наследника със същото име **скрива** полето на родителя
- Двете полета съществуват едновременно
- Кое се вижда зависи от **типа на референцията**
- Объркващо – **избягвайте** скриване на полета!

```
class Parent {
    String name = "Parent";
}

class Child extends Parent {
    String name = "Child";       // скрива

    void show() {
        System.out.println(name);         // Child
        System.out.println(super.name);  // Parent
    }
}

Parent p = new Child();
System.out.println(p.name);   // Parent!
```

03

Припокриване и полиморфизъм

Една референция – различно поведение

Припокриване на методи (override)

- Наследникът дефинира метод със **същата сигнатура**
- Версията на наследника **замества** тази на родителя
- Типът на връщане – същият или наследник (ковариантен)
- Достъпът не може да се **стеснява**
- `@Override` – компилаторът проверява, че наистина припокриваме

```
class Animal {
    public String sound() {
        return "...";
    }
}

class Dog extends Animal {
    @Override
    public String sound() {
        return "Бау!";
    }
}

class Cat extends Animal {
    @Override
    public String sound() {
        return "Мяу!";
    }
}
```



Винаги пишете @Override

Без анотацията грешка в името или параметрите (`tostring()` вместо `toString()`) създава **нов** метод, а не припокрива стария – и програмата тихо работи грешно.

Претоварване и припокриване

Претоварване (overload)

- същото име, **различни параметри**
- в един и същ клас (или наследник)
- изборът е при **компилиране**
- `print(int)`, `print(String)`

Припокриване (override)

- същото име и **същите параметри**
- в наследник
- изборът е при **изпълнение**
- `Dog.sound()` замества `Animal.sound()`

Полиморфизъм

- **Поли-морфизъм** = „много форми“
- Референция от тип родител може да сочи обект от тип наследник
- При извикване на метод се изпълнява версията на **реалния обект**
- Това се нарича **динамично свързване** (late binding)
- Кодът работи с общия тип и не зависи от конкретните наследници
- Нови наследници могат да се добавят, без да се променя съществуващият код

Полиморфизъм в действие

- Масивът е от тип `Animal`
- Всеки елемент отговаря по свой начин
- Цикълът **не знае** конкретните типове

```
Animal[] zoo = {
    new Dog(),
    new Cat(),
    new Dog(),
    new Animal()
};

for (Animal a : zoo) {
    System.out.println(a.sound());
}
// Бау!
// Мяу!
// Бау!
// ...
```

Как работи динамичното свързване?

- Компилаторът проверява, че методът съществува в **типа на референцията**
- При изпълнение JVM гледа **класа на реалния обект**
- Търси метода от този клас нагоре по йерархията
- Изпълнява първата намерена реализация
- За бързина JVM използва **таблица на виртуалните методи**
- `static`, `private` и `final` методите се свързват при компилиране

Пример: заплати в полиморфен списък

```
List<Employee> staff = List.of(
        new Employee("Иван", 2000),
        new Manager("Мария", 3000, 1500),
        new Employee("Петър", 2200));

double total = 0;
for (Employee e : staff) {
    double s = e.salary();              // кой salary()? — решава се при изпълнение
    System.out.printf("%-8s %8.2f%n", e.getName(), s);
    total += s;
}
System.out.printf("Общо: %.2f%n", total);
```

Ако утре добавим `Director extends Manager`, този код няма да се промени

Многостепенна йерархия

- `Director` наследява `Manager`, който наследява `Employee`
- Всяко ниво може да надгради поведението чрез `super`
- Полиморфният код от предишния слайд работи и с новия клас

```
public class Director extends Manager {
    private final double shares;

    public Director(String name, double base,
                    double bonus, double shares) {
        super(name, base, bonus);
        this.shares = shares;
    }

    @Override
    public double salary() {
        return super.salary() + shares * 0.1;
    }
}

Employee d = new Director("Ива", 5000, 2000,
                          10_000);
d.salary();    // 5000 + 2000 + 1000 = 8000
```

Полиморфни параметри

- Метод с параметър от тип родител приема **всеки наследник**
- Един метод вместо много претоварени
- Тип `Object` приема всичко – но губим информация

```
static void printPayslip(Employee e) {
    System.out.printf("%s: %.2f лв.%n",
            e.getName(), e.salary());
}

printPayslip(new Employee("Иван", 2000));
printPayslip(new Manager("Мария", 3000, 500));
// Иван: 2000.00 лв.
// Мария: 3500.00 лв.
```

Смяна на типа: upcasting

- **Към родител** (upcasting) – винаги безопасно и автоматично
- През референцията виждаме **само членовете на родителя**
- Обектът не се променя – само „погледът“ към него

```
Manager m = new Manager("Мария", 3000, 500);
Employee e = m;           // upcasting
Object o = m;             // също upcasting

e.salary();               // 3500 – на Manager!
// e.getBonus();          // грешка: Employee
                          // няма такъв метод
```

Смяна на типа: downcasting

- **Към наследник** (downcasting) – явно, с `(Тип)`
- Компилаторът позволява, но при изпълнение може да е грешно
- Грешен тип → `ClassCastException`
- Проверка преди това: `instanceof`

```
Employee e = new Employee("Иван", 2000);

Manager m = (Manager) e;
// -> ClassCastException при изпълнение!

if (e instanceof Manager) {
    Manager mm = (Manager) e;   // безопасно
    System.out.println(mm.getBonus());
}
```

Pattern matching за instanceof

- От Java 16: проверка и преобразуване в една стъпка
- Променливата-шаблон е видима само там, където проверката е вярна
- По-кратък и по-безопасен код

```java
Object o = getSomething();

// Старият начин
if (o instanceof String) {
    String s = (String) o;
    System.out.println(s.length());
}

// Новият начин
if (o instanceof String s) {
    System.out.println(s.length());
}

if (o instanceof String s && !s.isBlank()) {
    System.out.println(s.strip());
}
```

Припокриване и скриване на статични методи

- Обектните методи се **припокриват** – решава реалният обект
- Статичните методи се **скриват** – решава типът на референцията
- Затова извиквайте статичните методи през **името на класа**

```
class Animal {
    static String type() { return "животно"; }
    String name() { return "животно"; }
}

class Cat extends Animal {
    static String type() { return "котка"; }
    @Override String name() { return "котка"; }
}

Animal a = new Cat();
System.out.println(a.name());   // котка
System.out.println(a.type());   // животно!
System.out.println(Cat.type()); // котка
```

04

Класът Object

Методите, които всеки клас наследява

Методи на java.lang.Object

Метод	Предназначение
`toString()`	текстово представяне на обекта
`equals(Object o)`	логическо равенство
`hashCode()`	хеш код – за `HashMap`, `HashSet`
`getClass()`	класът на обекта по време на изпълнение
`clone()`	копие на обекта (`protected`, рядко се ползва)
`wait()`, `notify()`, `notifyAll()`	синхронизация между нишки
`finalize()`	**остарял**, не се използва

toString()

- По подразбиране: `ИмеНаКлас@хешКод` – безполезно
- Припокрийте го във всеки свой клас
- Използва се от `println`, конкатенация, дебъгер, логове

```
Employee e = new Employee("Иван", 2000);
System.out.println(e);
// Employee@1b6d3586

public class Employee {
    // ...
    @Override
    public String toString() {
        return "Employee[name=%s, salary=%.2f]"
                .formatted(name, baseSalary);
    }
}

System.out.println(e);
// Employee[name=Иван, salary=2000.00]
```

Договорът на equals()

- **Рефлексивност:** `x.equals(x)` е `true`
- **Симетричност:** `x.equals(y)` ⇔ `y.equals(x)`
- **Транзитивност:** от `x = y` и `y = z` следва `x = z`
- **Консистентност:** многократно извикване дава един и същ резултат
- `x.equals(null)` е винаги `false`
- По подразбиране `equals` сравнява **референции** (като `==`)

Правилна реализация на equals() и hashCode()

```
public class Point {
    private final int x, y;

    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

Правило: равни обекти **трябва** да имат равни `hashCode()` – иначе `HashSet` и `HashMap` не работят

hashCode() – защо е важен?

- Хеш таблиците търсят първо по хеш код, после с `equals`
- Ако припокрием само `equals` – „изгубени“ обекти
- `Objects.hash(...)` – удобен начин
- Записите (`record`) генерират и двата правилно

```
// Point с equals, но БЕЗ hashCode
Set<Point> set = new HashSet<>();
set.add(new Point(1, 2));

boolean found =
    set.contains(new Point(1, 2));
// false! – различни хеш кодове

// С правилен hashCode -> true
```

getClass() и clone()

- `getClass()` – реалният клас на обекта (връща `Class<?>`)
- `getSimpleName()` – кратко име
- `clone()` – копира обект; изисква `Cloneable`; труден за правилна употреба
- Предпочитайте **копиращ конструктор** или фабричен метод

```
Employee e = new Manager("Мария", 3000, 500);
System.out.println(e.getClass().getSimpleName());
// Manager

// Вместо clone():
public Point(Point other) {   // копиращ
    this(other.x, other.y);
}

Point copy = new Point(original);
```

05

Абстрактни и final класове

Непълни класове и забрана за наследяване

Абстрактни методи и класове

- **Абстрактен метод** – само декларация, без тяло
- Клас с абстрактен метод трябва да е **абстрактен**
- От абстрактен клас **не може** да се създаде обект
- Наследниците трябва да реализират абстрактните методи (или също да са абстрактни)

```
public abstract class Shape {
    private final String name;

    protected Shape(String name) {
        this.name = name;
    }

    public abstract double area();

    public String describe() {
        return name + ", лице = "
             + String.format("%.2f", area());
    }
}

// new Shape("?");  -> грешка
```

Наследник на абстрактен клас: Circle

- Реализира абстрактния метод `area()`
- Подава името на родителя чрез `super(...)`
- Наследява `describe()` готов

```
public class Circle extends Shape {
    private final double r;

    public Circle(double r) {
        super("Кръг");
        this.r = r;
    }

    @Override
    public double area() {
        return Math.PI * r * r;
    }
}

Shape s = new Circle(1);
System.out.println(s.describe());
// Кръг, лице = 3.14
```

Какво може да съдържа абстрактен клас?

Елемент	Позволено?
абстрактни методи (без тяло)	да
конкретни методи (с тяло)	да
полета, вкл. `private`	да
конструктори (извикват се от наследниците)	да
статични методи и полета	да
създаване на обект с `new`	не
абстрактен метод, който е `private`, `static` или `final`	не

Шаблонен метод (template method)

- Абстрактният клас дефинира **общия алгоритъм**
- Отделни стъпки са абстрактни – реализират ги наследниците
- Пример: `describe()` в `Shape` използва абстрактния `area()`
- Общото поведение е на едно място, различното – в наследниците
- Използва се широко в библиотеките на Java (напр. `AbstractList`)

final класове и методи

- `final` клас – **не може** да се наследява (`String`, `Integer`, `Math`)
- `final` метод – **не може** да се припокрие
- Защо? Сигурност, неизменяемост, гарантирано поведение
- Метод, извикван от конструктор, трябва да е `final` или `private`

```
public final class Money {
    private final long cents;

    public Money(long cents) {
        this.cents = cents;
    }
}

// class Euro extends Money { }  -> грешка

public class Account {
    public final String id() {
        return "ACC-" + hashCode();
    }
}
```

Типични грешки при наследяване

- Наследяване само заради преизползване на няколко метода
- Наследяване, при което „е“ не е вярно: `Stack extends Vector` в старата Java
- Извикване на припокриваем метод от конструктора на родителя
- Скриване на полета със същото име
- Забравен `@Override` или стеснен достъп
- Твърде дълбоки йерархии (повече от 3–4 нива)

Принцип на заместване на Лисков

- Обект на наследник трябва да може да **замести** обект на родителя, без програмата да се счупи
- Наследникът не бива да:
  - изисква повече от родителя (по-строги проверки на входа)
  - обещава по-малко (по-слаби гаранции за резултата)
  - хвърля неочаквани изключения
- Класически контрапример: `Square extends Rectangle` – `setWidth` променя и височината
- Ако наследникът „нарушава договора“ – използвайте композиция

Наследяване или композиция?

Наследяване (is-a)

- силна връзка с родителя
- промени в родителя чупят наследниците
- уместно, когато наистина „е вид“
- `Manager` е `Employee`

Композиция (has-a)

- обектът **съдържа** друг обект
- по-гъвкаво, по-лесно за тестване
- поведението може да се сменя
- `Car` има `Engine`

Композиция – пример

```
interface Engine {
    void start();
    int power();
}
class ElectricEngine implements Engine {
    public void start() { System.out.println("ззз"); }
    public int power() { return 150; }
}

class Car {
    private final Engine engine;     // has-a

    Car(Engine engine) { this.engine = engine; }

    void drive() {
        engine.start();
        System.out.println(engine.power() + " kW");
    }
}
```



Предпочитайте композиция пред наследяване

Принцип от **Effective Java**. Наследявайте само когато връзката „е“ е вярна и класът е проектиран за наследяване.

06

Sealed класове и pattern matching

Контролирани йерархии и switch по тип (Java 17–21)

Запечатани (sealed) класове

- `sealed` ограничава **кои** класове могат да наследяват
- `permits` – изброява разрешените наследници
- Всеки наследник е `final`, `sealed` или `non-sealed`
- Компилаторът **знае** всички възможни подтипове

```
public sealed interface Shape
        permits Circle, Square, Triangle { }

public record Circle(double r)
        implements Shape { }

public record Square(double side)
        implements Shape { }

public record Triangle(double a, double h)
        implements Shape { }
```

Pattern matching в switch (Java 21)

```
static double area(Shape s) {
    return switch (s) {
        case Circle c   -> Math.PI * c.r() * c.r();
        case Square sq  -> sq.side() * sq.side();
        case Triangle t -> 0.5 * t.a() * t.h();
        // няма default — компилаторът знае, че случаите са всички
    };
}

static String describe(Object o) {
    return switch (o) {
        case null                -> "нищо";
        case Integer i when i > 0 -> "положително число " + i;
        case Integer i           -> "число " + i;
        case String s            -> "низ с дължина " + s.length();
        default                  -> "друго: " + o;
    };
}
```

Record patterns – деконструиране

- Записът се „разглобява“ направо в шаблона
- Може да се влагат шаблони
- `_` – безименна променлива (Java 22+), когато стойността не ни трябва

```
record Point(int x, int y) { }

static String where(Object o) {
    return switch (o) {
        case Point(int x, int y)
                when x == 0 && y == 0
                -> "начало";
        case Point(int x, int y) when y == 0
                -> "по оста x";
        case Point(var x, var y)
                -> "(" + x + ", " + y + ")";
        default -> "не е точка";
    };
}
```

Пример: изрази като дърво

- `sealed` интерфейс + записи = дърво на израз
- `switch` по шаблони изчислява рекурсивно
- Нов вид възел → компилаторът показва къде да се добави

```
sealed interface Expr
        permits Num, Add, Mul { }
record Num(double v) implements Expr { }
record Add(Expr l, Expr r) implements Expr { }
record Mul(Expr l, Expr r) implements Expr { }

static double eval(Expr e) {
    return switch (e) {
        case Num n -> n.v();
        case Add(Expr l, Expr r) -> eval(l) + eval(r);
        case Mul(Expr l, Expr r) -> eval(l) * eval(r);
    };
}

// (2 + 3) * 4
eval(new Mul(new Add(new Num(2), new Num(3)),
             new Num(4)));     // 20.0
```

Защо е важно: проверка за пълнота

- При `sealed` йерархия `switch` може да е **без** `default`
- Ако добавим нов подтип, напр. `Hexagon`:
  - компилаторът показва грешка във **всеки** непълен `switch`
  - няма как да забравим да го обработим
- Съчетание: `sealed` интерфейс + `record` наследници + `switch` по шаблони
- Това е стил, близък до **алгебричните типове** от функционалните езици

Два стила на полиморфизъм

Обектно-ориентиран

- абстрактен метод `area()` в родителя
- всеки наследник го реализира
- лесно се **добавят нови типове**
- трудно се добавят нови операции

С pattern matching

- `switch` по типа на данните
- операцията е на едно място
- лесно се **добавят нови операции**
- новите типове изискват промени в `switch`

Обобщение

- `extends` – единично наследяване; коренът е `Object`
- Конструкторите не се наследяват; `super(...)` извиква родителя
- `@Override` + **динамично свързване** = полиморфизъм
- Upcasting е безопасен; downcasting – с `instanceof` (по шаблон)
- Припокривайте `toString`, `equals` и `hashCode` заедно
- `abstract` – непълни класове; `final` – забрана за наследяване/припокриване
- `sealed` + `record` + `switch` по шаблони – съвременен стил
- Предпочитайте композиция пред наследяване

Въпроси за самопроверка

- Каква е разликата между претоварване и припокриване?
- Какво ще стане, ако родителят няма конструктор без параметри?
- Кога възниква `ClassCastException` и как да го избегнем?
- Защо при припокриване на `equals` трябва да се припокрие и `hashCode`?
- Може ли абстрактен клас да има конструктор? А полета?
- Какво дава `sealed` в комбинация със `switch`?
- Кога бихте избрали композиция вместо наследяване?

Въпроси?

Следваща лекция: Интерфейси, вложени класове и ламбда изрази

